Description of all files in the allInstancesAPA folder

November 14, 2011

There are 3 folders in the allInstancesAPA folder: DIMACS, Culberson & Bipartite. The instances are stored separetely according to its class within each of those three folders. There are:

1. DIMACS (203 instances):
    - articial graphs
    - register allocation graphs
    - geometric graphs
    - random graphs
    - leighton graphs
    - flat graphs
2. NewRandom/Geometric graphs (540 instances)
    - geometric graphs (240 instances)
    - random graphs (300 instances)
3. Culberson graphs (@1000 instances, thus, 5000 instances in total):
    - Uniform or IID
    - Girth and degree inhibited
    - Geometric
    - Weight-biased Graphs
    - Cycle Driven Graphs
4. Bipartite graphs (1000 instances)

The description of each class of instances are as follow:

1 DIMACS/Random/Geometric Graphs

In this folder, there are 7 classes of instances. There are:

1. **Artificial Graphs.** These instances are DIMACS benchmarks instances that are taken from ftp://dimacs.rutgers.edu/pub/challenge/graph/. These include: Anna, David, Huck, Jean, Homer, Games120, miles250, miles500, miles750, miles 1000, miles 1500, myciel3, myciel4, myciel5, myciel6, myciel7, queen5_5, queen6_6, queen7_7, queen8_8, queen9_9 and queen10_10. There are 22 instances in total.

2. **Register Graphs**. These are also DIMACS benchmarck instances. These include mulsol.i, zeroin.i, inithix.i and fpsol.i. There are 14 instances in total.

3. **Geometr**ic **Graphs**. The graph is denoted by g (n; d) where the number of vertices is denoted by the rst number while the density d is denoted by the second number. All random graphs are randomly generated using built-in generator from an open source Python package called Networkx that is available at http://networkx.lanl.gov/reference/. We generated graphs with 100, 150, 200, 250 nodes with densities 0.1, 0.5, 0.9. We generated 5 instances for each type of instances. There are 4 x 3 x 5 = 60 instances in total.

4. **Random Graphs**. The graph is denoted by G(n; p) where the number of vertices is de- noted by the rst number while the probability p is denoted by the second number. All random graphs are also randomly generated using built-in generator from an open source Python package called Networkx. The generator are available at http://networkx.lanl. gov/reference/. We generated graphs with 30, 50, 70 nodes with edge probabilities 0.1, 0.3, 0.5, 0.7, 0.9. We generated 5 instances for each type of instances. There are 3 x 5 x 5 = 75 instances in total.

5. **Large Random Graphs**. The structures of these large random graphs are the same with the small one, except the number of nodes are larger, 125 to 500 nodes, and the chromatic number of these instances are unknown. These include DSJC125.1, DSJC125.5, DSJC125.9, DSJC250.1, DSJC250.5, DSJC250.9, DSJC500.1, DSJC500.5, DSJC500.9, DSJR500.1 and DSJR500.5. The last two graphs are reverse random graphs. There are 14 instances in total.

6. **Leighton Graphs**. The graphs are generated by a procedure that uses the number of vertices, the desired chromatic number, the average vertex degree and a random vector of integers to generate a certain number of cliques. These include le450_5a, le450_5b, le450_5c, le450_5d, le450_15a, le450_15b, le450_15c, le450_15d, le450_25a, le450_25b, le450_25c and le450_25d. The total of at graphs is 12 instances.

7. **Flat Graphs**. They are generated by partitioning the set of vertices into Kp almost equal sized classes and then by selecting edges only between vertices of dierent classes. Finding the best legal K-coloring is equivalent to the restoration of the initial partitioning and K = Kp is hence the chromatics number of the graph (denoted by the second number). These include at300_20, at300_26, at300_28, at1000_50, at1000_60 and at1000_76.

2 NewRandom/Geometric Graphs

In this folder, we generated random and geometric graphs that have the same structures with the previous random/geometric graphs, and this time we generated 20 more instances for each type of graphs. Thus, there are 540 instances in total (for random & geometric). There are:

1. **Geometric Graphs**. The graph is denoted by g (n; d) where the number of vertices is denoted by the rst number while the density d is denoted by the second number. All random graphs are randomly generated using built-in generator from an open source Python package called Networkx that is available at http://networkx.lanl.gov/reference/. We generated graphs with 100, 150, 200, 250 nodes with densities 0.1, 0.5, 0.9. We generated 5 instances for each type of instances. There are 4 x 3 x 20 = 240 instances in total.

2. **Random Graphs**. The graph is denoted by G(n; p) where the number of vertices is de- noted by the rst number while the probability p is denoted by the second number. All random graphs are also randomly generated using built-in generator from an open source Python package called Networkx. The generator are available at http://networkx.lanl. gov/reference/. We generated graphs with 30, 50, 70 nodes with edge probabilities 0.1, 0.3, 0.5, 0.7, 0.9. We generated 5 instances for each type of instances. There are 3 x 5 x 20 = 300 instances in total.

3 Culberson Graphs

There are five classes for Culberson graphs, and we generated 1000 instances for each class. Thus, the total is 5000 instances for the Culberson graphs. The five instances generators available from Culberson's website and the class are described as follow.

1. **Uniform or IID**: edges are assigned to vertex pairs with a fixed probability p

2. **Girth and degree inhibited**: Each graph is assigned a probability p, girth limit g; and a degree limit .

3. **Geometric**: These graphs are generated by choosing a radius r and uniformly distributing n pairs of numbers (x; y) in the range of 0 x; y $<$ 1.

Vertices in the graph correspond to the points (x; y) in the plane, with edges included if the distance between a pair of vertices is less than the radius r.

4. **Weight-biased Graphs**: These graphs contain cliques limited to a given size. Each clique is generated by randomly creating h color partitions, then randomly selecting one of the vertices in each partition and joining every pair by an edge. Each clique is generated independently. Such restrictions on the size of the cliques reduces large variations in structure that might aid an algorithm like DSATUR.

5. **Cycle Driven Graphs**: Similar to the weight-biased graphs, this generator creates cycles of a specied length. h color partitions are created, the algorithm then generates a cycle by randomly generating a path with each vertex from a dierent color partition than the last.

4 Bipartite Graphs

We generated 1000 instance for bipartite graphs, and each file is written in DIMACS format.